a

James Wang

October 2026

Our greedy solution is, for every unassigned trip, find the car that provides them the minimum trip time. Then, from this list of trips with a car each, pick the one with the longest trip time. Suppose we currently have n passengers and m cars. Some of the cars could currently be en route with another passenger to their destination. This why we use our formula $\max\{e + d(w', s) - t,\ 0\}$.

Now although this is not optimal since it is NP-hard, we can provide a greedy approxmination to show that it is a proportion of optimal. We can start by assuming some optimal solution **OPT** where OPT is the true optimal solution for each trip and car.

Our simplification is that we have combined the wait and journey times together, so we are essentially minimising the total trip time combined.

Suppose we have a car c that has been determined to be the car assigned to a trip t. Now for t in any single case, the trip time is always at least the amount of time it takes for the car to arrive + journey time to travel there, so $OPT \geq w_t + j_t$, where w is the travel time it takes for the car to pick up the passenger, and j is the journey time.

Now for t in a volume case, where there are many trips scheduled before our own trip t for any car c out of a pool of selection of cars, the trip time is always at least the amount of time it takes for the cars to both arrive and carry out the journey for each of the already scheduled trips beforehand. Hence $OPT \geq \frac{1}{m}(\sum_{i=1}^{n} w_i + j_i) - (r_t - r_1)$, where j is the journey time and n is the total number of already queued trips, as if a car was not busy, they would simply be able to just go straight for our request t, like in the single case. m is the total number of cars, and hence we divide the total work by m. $r_t - r_1$ is the total number of time that has elapsed between our trip request and the very first trip request. This is because cars will have had time for a 'head start' between the two requests, so we need to subtract off this time. Note here there is no extra wait time w_t or journey time j_t, as the car has not been allocated yet and hence w_t and j_t do not exist.

Now to find our upper approximation bound, we know that currently, and up until this point, all m cars have been in operation and occupied in current trips. This is because otherwise, under our greedy heuristic, if there was still a free unoccupied car, this car would get allocated towards our longest bottleneck trip, which is our upper bound we are trying to calculate. Hence we can start with the total time all the cars have been occupied with the formula $(m -$

$1)(r_t + w_t + j_t - r_1) + 1(r_t - r_1)$ which is just the number of cars multiplied by the minutes between the first and current requests, and the time for one car stopping as soon as the request is received. $m - 1$ cars are still working while the 1 car is completing its allocated trip. This value has to be less than the total amount of time spent on the trips and travel time combined, hence $(m-1)(r_t + w_t + j_t - r_1) + 1(r_t - r_1) \leq (\sum_{i=1}^{k} w_i + j_i)$. We care about the overall time between the moment the trip request is received up until the car completed its trip (picked up the passenger and completed the journey). We can now also denote this as c_t. Hence $c_t = r_t + w_t + j_t$. Rearranging for r_t and substituting this into our inequality, we get

$$
\begin{aligned}
(m-1)(c_t - w_t - j_t + w_t + j_t - r_1) + 1(c_t - w_t - j_t - r_1) &\leq (\sum_{i=1}^{k} w_i + j_i) \\
(m-1)(c_t - r_1) + c_t - w_t - j_t - r_1 &\leq (\sum_{i=1}^{k} w_i + j_i) \\
m(c_t - r_1) - w_t - j_t &\leq (\sum_{i=1}^{k} w_i + j_i) \\
c_t - r_1 &\leq \frac{1}{m}(\sum_{i=1}^{k} w_i + j_i) + \frac{1}{m}(w_t + j_t)
\end{aligned}
$$

Now we want to bring in our OPT bounds from earlier, so we have to perform some algebraic manipulation, first by subtracting $r_t - r_1$ from both sides.

$$
\begin{aligned}
c_t - r_1 &\leq \frac{1}{m}(\sum_{i=1}^{k} w_i + j_i) + \frac{1}{m}(w_t + j_t) \\
c_t - r_1 - (r_t - r_1) &\leq \frac{1}{m}(\sum_{i=1}^{k} w_i + j_i) - (r_t - r_1) + \frac{1}{m}(w_t + j_t) \\
c_t - r_t &\leq \frac{1}{m}(\sum_{i=1}^{k} w_i + j_i) - (r_t - r_1) + \frac{1}{m}(w_t + j_t)
\end{aligned}
$$

As we can see we now have our trip time on the LHS and we can group our terms on the RHS to match our OPT bounds from earlier, as we are trying to achieve our maximum upper bound for $c_t - r_t$. Substituting in our two bounds, we get

$$
\begin{aligned}
\max(c_t - r_t) &\leq OPT + OPT\left(\frac{1}{m}\right) \\
\max(c_t - r_t) &\leq OPT\left(1 + \frac{1}{m}\right)
\end{aligned}
$$

and hence this is our approximation bound for our min-max algorithm where it is at most $\left(1 + \frac{1}{m}\right)$ times worse than the optimal solution.